A

Project Synopsys

on

YAARA - TRAVELOGUE STUDIO: A WEB-BASED PHOTOBOOK EDITOR AND PREVIEW PLATFORM

Submitted in partial fulfillment for the award of degree of

Bachelor of Technology

In

Computer Science & Engineering

Submitted to:
Dr. Reema Amera
Professor

Submitted By:
Roll No. - Student Name
Roll No. - Student Name
Roll No. - Student Name
Roll No. - Student Name

Department of Computer Science & Engineering

Global Institute of Technology

Jaipur (Rajasthan)-302022

Batch: 2023-27

Yaara - Travelogue Studio: A Web-Based Photobook Editor And Preview Platform

Content:

Problem Statement

Objectives

Proposed Methodology

Technologies/Tools to be Used

Expected Outcomes

1. Problem Statement

Photographs are captured in large numbers, but converting them into a coherent, printable travel photobook remains a time-consuming task. Users often have to switch between file managers, generic design tools, and PDF utilities to select images, arrange layouts, add captions, preview pages, and prepare an export. These tools usually demand design experience and make it difficult to maintain visual consistency across an entire book.

The project addresses this gap through Yaara (Travelogue Studio), a browser-based photobook creation system focused on a short, practical workflow: choose templates, upload photographs, fill pages, customize the design, preview the book, and export it. The system must protect administrator-authored templates while still allowing users to replace photographs and edit permitted content. It must also preserve user intent during automation: already-filled frames are not overwritten, excluded photographs remain excluded, and empty locked frames are unlocked only when a fill action needs them.

A further challenge is reliable project continuity. Photographs and project data need local persistence during editing, while production template metadata and shared media require secure cloud storage. The same editable project title and stable template identifiers must remain consistent in the editor, preview, PDF export, and .wanderbook project-file flows.

2. Objectives

- Provide a fast, template-first workflow for creating a complete square photobook from uploaded travel photographs.
- Allow users to select one or more categorized templates, preserve their chosen order, and open the resulting pages directly in the editor.
- Support photo upload, drag-and-drop placement, crop, zoom, rotation, filters, frames, shape masks, captions, stickers, text, drawings, and customizable page backgrounds.
- Implement Magic Fill to populate empty frames across the whole book without replacing existing photographs or using excluded images.
- Protect administrator-created page structures, backgrounds, and locked elements while keeping user-created layouts flexible.
- Provide undo/redo, page management, responsive editing controls, realistic book preview, shareable preview support, PDF export, and compatible project saving/loading.
- Use secure production persistence in which Supabase stores structured metadata and ImageKit stores media, with private credentials restricted to server-side functions.

3. Proposed Methodology

The system follows a component-based, state-driven web architecture. TanStack Start supplies routing and server-function support, while React and TypeScript implement the landing page, editor, preview, administration, and reusable photobook components. The application state is centralized in a Zustand store and augmented with Zundo history so editing commands can participate in undo and redo.

The user begins on a compact template-discovery interface. Templates are loaded by category, previewed in a square card, and added to an ordered selection bucket. When the user proceeds, the selected template data is converted into book pages while retaining stable template IDs and protection metadata. Photographs are then uploaded into a library, with binary image data persisted locally through IndexedDB and lightweight editor state persisted through browser storage.

During editing, each page is represented as structured data containing a background and typed elements such as photos, stickers, text, quotations, and drawings. Photo elements retain position, size, rotation, crop offsets, zoom, opacity, filter settings, frame style, shape mask, captions, lock state, and optional background-removal or magic-mask data. The editor renders these elements on a square 5.5-inch page model and exposes page, design, library, and element-level controls.

Automation is implemented as deterministic store actions. Magic Fill scans the entire book for empty photo frames, selects only non-excluded library images, keeps existing frame assignments unchanged, and unlocks a locked frame only when that empty frame is filled. Magic Layout can derive a masked photo slot from a template or background image. Client-side subject segmentation supports background removal, with erase and restore refinement for the resulting mask.

For output, the preview route renders the pages as an interactive page-flipping book. PDF export uses jsPDF and canvas rendering to reproduce page backgrounds, crop and transform settings, masks, filters, frames, text, stickers, drawings, captions, and overlays. Production administration and shared-preview functions use server-side APIs; Supabase holds structured records and ImageKit holds uploaded media so private keys never reach the browser.

Verification is carried out through TypeScript compilation, linting, production builds, and manual responsive checks at desktop and narrow mobile widths. Functional tests focus on template selection order, protected-template behavior, upload persistence, Magic Fill safeguards, undo/redo, project compatibility, preview accuracy, and PDF output.

4. Technologies/Tools to be Used

- Frontend framework: React 19 with TanStack Start, TanStack Router, and Vite.
- Programming language and validation: TypeScript and Zod.
- User interface: Tailwind CSS 4, Radix UI primitives, lucide-react icons, and responsive CSS.
- State and history: Zustand for centralized photobook state and Zundo for undo/redo history.
- Interactive editing: dnd-kit for drag-and-drop, react-rnd and resizable panels for positioning and resizing, and browser Canvas APIs for rendering and masks.
- Preview and export: react-pageflip for book presentation, html2canvas-pro where DOM capture is required, and jsPDF for downloadable PDF generation.
- Image processing: @imgly/background-removal for client-side foreground segmentation, with custom crop, filter, shape-mask, and compositing logic.
- Persistence and deployment: IndexedDB and browser storage for local work; Supabase for production metadata; ImageKit for production media; Vercel for deployment.
- Development quality tools: ESLint, Prettier, TypeScript compiler, npm, and Git.

5. Expected Outcomes

- A working responsive web application that enables users to create a polished travel photobook from template selection through final export.
- A streamlined editor that supports multi-page books, rich photo styling, text and decorative elements, backgrounds, drawings, project titles, and reusable templates.
- Reliable automated filling that saves time while respecting filled frames, excluded photographs, and template locks.
- A realistic interactive preview and a high-quality PDF whose page content closely matches the editor, including crop, masks, filters, frames, stickers, text, and drawings.
- Persistent projects that can be resumed locally and remain compatible with saved .wanderbook files and stable template identifiers.
- A protected administration workflow for publishing and maintaining templates, categories, backgrounds, stickers, overlays, and thumbnails without exposing private cloud credentials.
- An extensible architecture that can later support collaboration, print-service integration, additional book sizes, advanced asset search, and richer sharing controls.